

Latent Preference-Based LLM Shopping Agent

Eileen Cui, Mina Dao, Franklin Huang, Karis Park, Lyann Sun
{ailincui, minadao, huafrank, karisp, lyannsun}@umich.edu

Abstract

LLM-based shopping assistants typically rely on explicit user instructions, often failing to infer underlying preferences that drive consumer behaviour. We developed a hybrid agent that actively uncovers customer preferences through strategic dialogue by integrating the curiosity-driven questioning of **CURIO** with the continuous latent space modeling of **VPL**. The agent maintained a multivariate Gaussian belief over eight preference dimensions, utilizing uncertainty-driven questions to refine its internal state before recommending products from the 10,000 item Webshop catalog. We compared our hybrid agent against the standalone CURIO and VPL baselines. Our agent significantly outperformed CURIO, achieving 75.2% **Success@1000** and a normalized regret (0.0508). These results demonstrate that combining latent variable modeling with uncertainty-driven questioning enables more precise preference alignment in high-density product environments.

1 Introduction

Shopping assistants powered by large language models (LLMs) have gained popularity in online retail platforms. As product catalogs grow, these assistants help navigate the overwhelming choices by interpreting queries, filtering products and comparing products quickly, allowing customers to reduce the time required to shop. Some of these shopping assistants can help you find the best deals or discounts, helping users save money. Furthermore, when integrated with platform data, shopping assistants can use browsing and purchase history to provide tailored recommendations that can improve customer satisfaction.

However, despite these benefits, these systems typically rely on explicitly stated instructions or surface-level information rather than inferring the underlying preferences that shape user decisions. In contrast, human shopping assistants can often detect underlying qualities such as style, risk tolerance, brand loyalty and aesthetic preferences by asking targeted questions and gradually understanding the customer’s tastes. Therefore, without truly understanding these attributes, the LLM-powered shopping assistants cannot provide the kind of shopping guidance human assistants are able to provide.

In this project, we aim to build an LLM-based agent that can achieve this capability- an agent that can identify the underlying customer preferences, ask strategic questions, update the internal user model and then use this model to guide product search and recommendations. Building this agent would enable a higher level of user personalization and lead to more accurate preference-aligned recommendations.

2 Related Work

Traditional training methods like Reinforcement Learning from Human Feedback (RLHF) are effective for aligning AI foundation models to human values, but they typically assume that all users share the same preference structure. A recent (2024) paper approaches this problem by formulating the training as a latent-variable problem, allowing to recover multi-modal preference distributions

that more effectively account for individualized utilities [2]. However, this approach is limited by the available data, training on feedback from users ahead of time.

In parallel, research has shifted from static product retrieval such as WebShop [5] to complex, multi-turn interactions. While ShopSimulator [1] highlights that agents struggle to consistently apply user profiles during dialogue, models like PEARL [7] address this by extracting explicit preferences from conversations. However, these systems remain limited to stated attributes.

In another study, the CURIOSITY framework introduces uncertainty-guided questioning, using a belief distribution over hidden user types as a signal to request clarification [4]. While effective for general personalization, CURIOSITY models uncertainty over discrete user types rather than maintaining a continuous latent preference representation that evolves throughout the interaction.

Our work builds on these previous works by adopting the latent-space perspective of VPL with the uncertainty-aware elicitation strategies of CURIOSITY. We address their limitations through online inference over targeted questioning and continuous latent preferences, allowing the system to reason about constraints that shape choices even when not explicitly stated, similar to "active reasoning" techniques used in task-oriented assistants [3]. Furthermore, we differentiate our work by treating under-specification as a trigger for clarification rather than a cue for a guess, balancing task efficiency with the precision required for long-horizon decision making in e-commerce.

3 Method

3.1 Data

We use the Webshop catalog [6], which contains 10,000 items, as the product pool in our shopping recommendation setting. Each item is represented by an 8-dimensional product embedding aligned with 8 latent preference factors: comfort, functionality, maintenance, status symbol, modesty, sustainability, risk tolerance, and trend sensitivity.

We simulated users with 20 separate attributes, such as age, gender, occupation, etc, for the models to ask about and uncover. These attributes have various mappings to mean and concentration parameters for different beta distributions. We then sample our 8 latent variables from these different beta distributions to obtain values between 0 and 1 for the eight preference factors. This is done to inject noise into our simulated users to reflect the random rather than deterministic nature of real world preferences.

We split the simulated users into 1,000 training users, 125 evaluation users, and 125 testing users.

3.2 Task Formulation

The main goal is to build an agent that can provide a personalized item recommendation by managing three internal states: a preference belief state that holds and refines its hypotheses about what the user truly values; a dialogue history which records the full context of the interaction; and a set of uncertainty metrics, which measures its confidence in the hypotheses. This integrated architecture allows the agent to act as an active investigator deliberately seeking information to resolve its own uncertainty.

To contextualize the performance of our agent, we compare it against two other established methods: Curiosity-driven User-modeling Reward as an Intrinsic Objective (CURIOSITY) and Variational Preference Learning (VPL) [2][4].

Each model is trained, validated and tested using the same generated simulated users and Webshop product catalog for search and recommendation [6]. To compare model performance, we define a utility function that measures how much the simulated user "likes" the item based on the latent features. Utility is computed as the negative squared Euclidean distance between

the product’s latent feature vector and the user’s latent preference vector. The product with the highest utility represents the product that best matches the user’s true preferences. With this utility function we can then compute the following metrics:

- Regret: the difference between the utility of the best product and the utility of the product recommended by the agent. Normalized regret is the regret divided by the maximum possible regret in that episode.
- Success@k: equals 1 if the agent’s recommended product is among the user’s top-k most liked products based on utility; 0 otherwise.
- Threshold success rate: equals 1 if recommended product is within the 70th percentile of utilities; 0 otherwise.
- Uncertainty reduction: measures how much a model reduces uncertainty about the user after each interaction turn.

3.3 CURIO

In the CURIO model, preference elicitation is framed as a reinforcement-learning problem in which the agent maintains a discrete belief distribution over user archetypes and asks questions that help map the user responses to the latent preference categories.

This model consists of a frozen MiniLM-based belief encoder that processes a tokenized conversation history to update the internal belief distribution over eight predefined user types that are based on the eight latent attributes. This belief distribution is then used directly by an actor-critic policy, where the actor selects the next latent preference question to ask and the critic estimates the expected future reward from the current belief state. CURIO is optimized using a composite reward function, consisting of an intrinsic reward that encourages the agent to ask more "informative" questions through Shannon entropy reduction, and an extrinsic reward, which reflects the quality of the final recommendation. During training, each episode was limited to a maximum of eight turns, and the model used a dual-optimizer setup: Adam with a learning rate of 5×10^{-3} for the belief heads and Adam with a learning rate of 3×10^{-4} for the actor and critic. At the end of each episode, the agent performs belief-weighted scoring across the Webshop catalog and selects the product that yields the highest utility based on the final belief distribution.

3.4 VPL

VPL formulates preference learning as a probabilistic inference problem, where the objective is to recover a user-specific latent representation from observed feedback.

The model maintains a continuous belief over an 8-dimensional preference vector and updates this belief as new evidence is observed. The model consists of a variational encoder that maps a collection of pairwise comparisons to a Gaussian posterior over the latent preference space. Each comparison (i, j, y) , indicating the preferred item, provides evidence about the user’s underlying utility function. The inferred posterior is then used by a latent-conditioned scoring function to assign scores to candidate items. Model training maximizes a pairwise likelihood objective derived from the Bradley–Terry model, combined with a Kullback-Leibler (KL) divergence term to regularize the latent distribution. Parameters are optimized using Adam with a learning rate of 3×10^{-4} , and model selection is based on validation regret. During inference, the model incrementally updates the posterior as additional comparisons are observed. After the feedback budget is exhausted, the final posterior is used to score the full candidate set, and the highest-scoring item is selected as the recommendation.

3.5 Our Agent (CURIO + VPL Hybrid)

For our agent, we combined aspects of both the CURIO model and the VPL model for a unified, hybrid approach.

Similar to VPL, our agent stores and updates each users’ preferences as two individual vectors of μ and σ^2 representing the mean and variance parameters for each of the underlying user attributes.

Uncertainty Estimation: μ is determined by our belief encoder, trained on the conversation history. σ^2 we wanted to reflect the uncertainty of the belief encoder. As a result, σ^2 is the sample variance of the output of the belief encoder when using Dropout($p = 0.3$) and a sample size of 5. Note that this means we utilize dropout in all stages training, eval, testing, because we need an uncertainty metric for the agent to decide on questions to ask.

We then included the composite reward function from CURIO, providing the agent with a reward for asking “informative” questions. By using a multivariate gaussian, we removed the need to define discrete user types. As a result, we defined “informative” questions as questions that reduced the variance of the model’s latent variable vector.

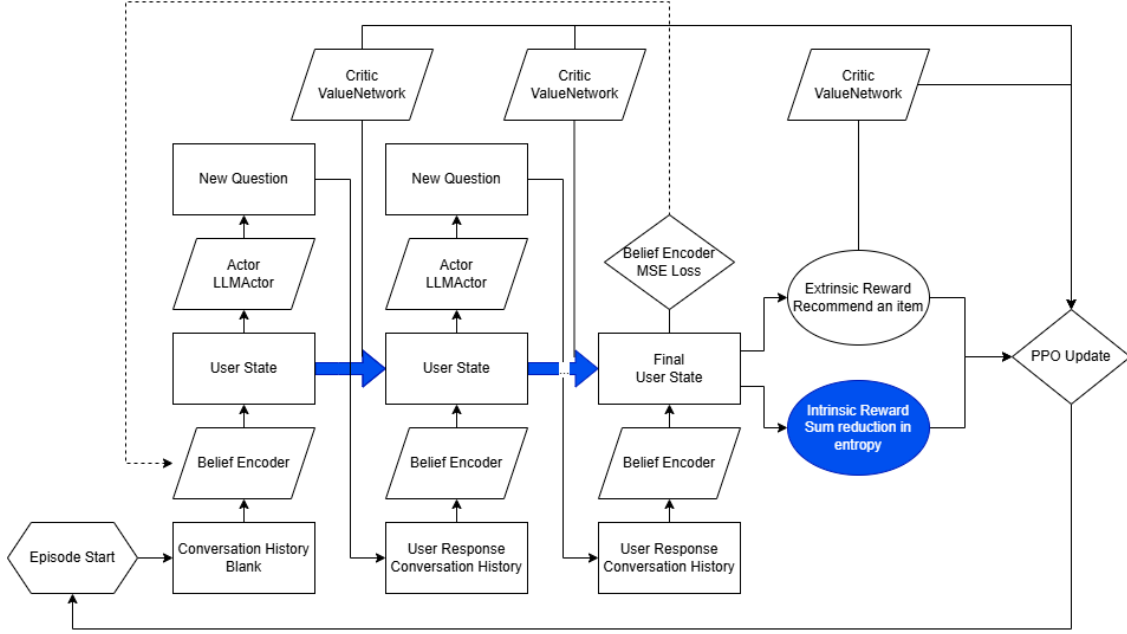


Figure 1: Combined Model Pipeline. Slightly simplified at the end as we batched together 25 episodes per ppo update.

After each user response, the history is tokenized and passed through the belief encoder to update the parameters, an updated μ and σ^2 . At each turn we can then calculate a reward based on the change in σ^2 to incentivize the model into asking informative questions, similar to the CURIO approach.

Initial Belief State:

$$\begin{aligned}\mu_0 &= [0.6049 \quad 0.7046 \quad 0.7361 \quad 0.4230 \quad 0.5068 \quad 0.5659 \quad 0.4841 \quad 0.4974] \\ \sigma_0 &= [0.0493 \quad 0.1047 \quad 0.0451 \quad 0.0624 \quad 0.0198 \quad 0.0251 \quad 0.0288 \quad 0.0359]\end{aligned}$$

Model: “Could you tell me more about your income?”

User: *“I’m honestly earning just enough to cover the basics, like rent and groceries, and it’s pretty tight.”*

Updated Belief State:

$$\begin{aligned}\boldsymbol{\mu}_1 &= [0.6404 \quad 0.7022 \quad 0.6987 \quad 0.3481 \quad 0.4824 \quad 0.5411 \quad 0.4555 \quad 0.5005] \\ \boldsymbol{\sigma}_1 &= [0.0180 \quad 0.0738 \quad 0.0355 \quad 0.0536 \quad 0.0138 \quad 0.0313 \quad 0.0236 \quad 0.0249]\end{aligned}$$

The belief distribution at each turn of $\boldsymbol{\mu}$ and σ^2 feeds into a learned policy distribution to decide on a question for the next turn. The policy is trained to maximize the composite reward function, a combination of both r_{int} at each turn and r_{ext} from the final product recommendation.

Our reward function for the model follows the same composite reward function as CURIO, with a split between intrinsic and extrinsic rewards. The intrinsic reward represents the gain in information between each turn, or the reduction in entropy between each turn. Since we are representing the latent variables with a multidimensional gaussian, entropy at each turn is calculated as follows from the gaussian entropy:

$$H = \frac{1}{2} \sum_i \log(2\pi e \sigma_i^2)$$

And for the reward at each turn we do $r_{int} = H_{before} - H_{after}$.

The extrinsic reward is calculated at the final step using the recommendation output. Our model selects the recommendation with the largest utility under the current belief distribution’s mean, where utility is calculated as distance between the mean vector and the product embedding: $p^* = \arg \max_p -\|p - \boldsymbol{\mu}\|^2$. Using p^* , we calculate $r_{ext} = -\|p^* - z_{true}\|^2$. The total reward for each episode is the extrinsic reward plus the sum of the intrinsic rewards times alpha:

$$r_{total} = r_{ext} + \alpha \sum r_{int}$$

Using this reward structure, we train our three networks with two objective functions. The belief encoder is trained with MSE loss against the true user latent vector z^* :

$$\mathcal{L}_{enc} = \frac{1}{T} \sum_{t=1}^T \|\boldsymbol{\mu}_t - \mathbf{z}^*\|^2$$

For the actor and critic, we optimize a PPO objective combining the clipped policy gradient with the value network loss:

$$\mathcal{L}_{PPO} = - \sum_{t=1}^T \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) + \frac{1}{2} \sum_{t=1}^T (G_t - V(s_t))^2$$

where $r_t(\theta)$ is the probability ratio between the new and old policies, $\hat{A}_t = G - V(s_t)$ is the advantage estimate representing how much better or worse an action was relative to the critic’s expectation, and $\epsilon = 0.2$ is the clipping parameter that prevents overly large updates that destabilize training.

4 Results

4.1 CURIO

An analysis of the 1000-episode training process indicates a successful transition from stochastic exploration to policy stability. The model effectively transcended the initial reward floors, with

smoothed cumulative rewards stabilizing between 0.4 and 0.7 (Appendix A: Figure 3). This trend suggests the reinforcement learning objective successfully incentivized the selection of high-utility actions.

Empirical evaluation on a held-out user set demonstrated a mean belief accuracy of 21.6%. While the policy relies on the full 8-turn dialogue window—as evidenced by the lack of early convergence at turn 5—the consistent average entropy reduction of 0.206 validates the ability of the LLM-driven questions to successfully extract latent preference vectors (Appendix A: Figure 4).

Product retrieval performance demonstrates a high degree of "coarse-grain" precision. The policy achieved a Success@500 of 57.6% and a Success@1000 of 68.0%, placing optimal items within the top tier of a high-density catalog (Appendix A: Figure 6). The model also reached a threshold success rate of 84.0%, maintaining a low normalized regret of 0.062 (Appendix A: Figure 5). This signifies that the utility loss relative to an Oracle is minimal on unseen distributions.

4.2 VPL

VPL achieves strong utility-based performance under the shared latent-space recommendation setting. As shown in Table 2, it attains a low average regret of 0.0186 and an average normalized regret of 0.0588, indicating that the recommended items are often close in utility to the oracle-best product. Training dynamics further show that the model converges reliably, with validation regret stabilizing after the early epochs (Appendix B: Figure 7).

VPL also shows consistent posterior refinement across sequential interactions, with an average uncertainty reduction of 0.776 per turn. This behavior is aligned with the variational formulation of the model, in which each additional pairwise comparison contributes evidence for updating the latent preference posterior, as illustrated by the decreasing posterior entropy across turns (Appendix B: Figure 8).

This behavior is also reflected in the regret distribution, where most test episodes remain concentrated in the low-regret region (Appendix B: Figure 9). In terms of ranking behavior, VPL performs substantially better at larger cutoffs than at the exact top of the list. While Success@100 is 0.288 and Success@200 is 0.336, the corresponding values increase to 0.560 for Success@500 and 0.736 for Success@1000. This pattern indicates that VPL is generally successful at placing relevant items in the upper portion of the ranked list, even though identifying the single best item remains difficult in a large candidate space (Appendix B: Figure 10).

Overall, these results suggest that VPL can infer a coherent latent preference representation from pairwise feedback and translate it into reasonable large-scale ranking performance.

4.3 Our Agent (CURIO + VPL Hybrid)

Our agent was also trained on 2000 episodes. The table below shows some metrics we monitored while training the model: the rewards, the uncertainty of our model at the start of each episode, and MSE metrics of the estimated latent variables at different points during each episode.

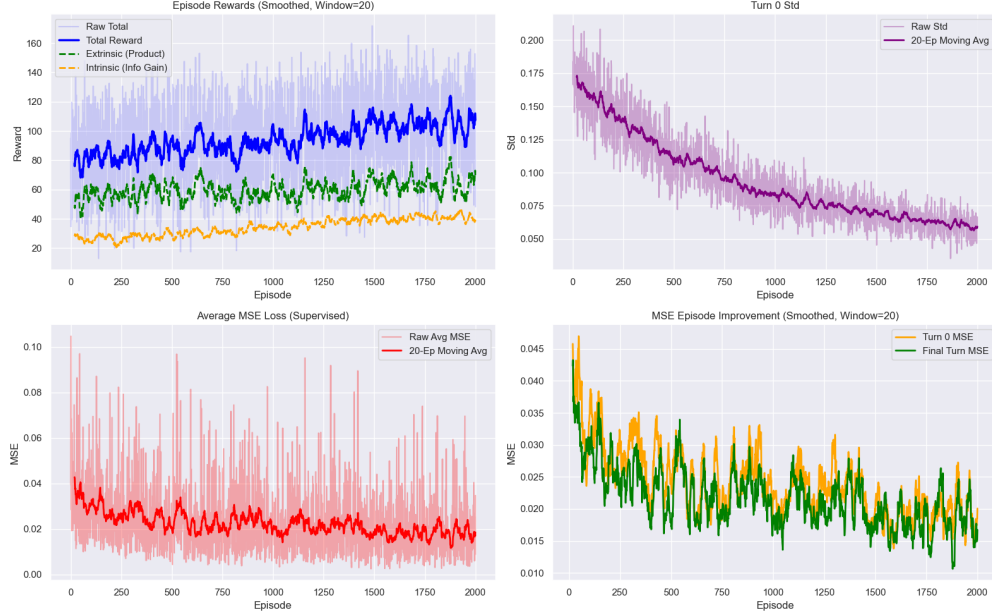


Figure 2: Model Training Diagnostics.

The decrease in Turn 0 Std and reduction in Turn 0 MSE (yellow line in bottom right graph) shows that our belief encoder is slowly learning and becoming more confident in the “average” user latent variables in the test set. However, the slow increase in intrinsic reward (yellow line in top left graph) shows that the combined model seems to also be learning simultaneously what questions to ask to fine-tune the user latent variables instead of just sticking to the average user. The questions the model asks the user reflect this, with the model consistently identifying and asking about 8/10 attributes that impact the latent variables.

Additionally we conducted an ablation study to identify the impact of removing either the extrinsic reward or the intrinsic reward from our model. The results show that even without the extrinsic or intrinsic reward helping the actor select which specific attributes to ask about, the belief encoder is able to perform well enough to learn an “average” user latent variable vector so the actor can follow a safe policy of recommending to the average user. It seems that removing the intrinsic rewards results in worse model performance than removing extrinsic rewards.

Table 1: Combined Model Ablation Study

Metric	Remove Extrinsic	Remove Intrinsic	Full Model
Avg. Regret	0.1472	0.1599	0.1342
Avg. Normalized Regret	0.0566	0.0618	0.0508
Success@100	0.016	0.032	0.064
Success@200	0.016	0.032	0.072
Success@500	0.584	0.568	0.592
Success@1000	0.712	0.712	0.752
Avg. Uncertainty Reduction Per Turn	0.4724	*	0.4220

4.4 Summary of the results across models

Table 2: Cross Model Performance Comparison

Metric	CURIO	VPL	Our Agent
Avg. Regret	0.1675	0.0186	0.1342
Avg. Normalized Regret	0.0651	0.0588	0.0508
Success@100	0.015	0.288	0.064
Success@200	0.1	0.336	0.072
Success@500	0.57	0.560	0.592
Success@1000	0.7	0.736	0.752
Avg. Uncertainty Reduction Per Turn	0.0282	0.776	0.4220

From Table 2, our agent demonstrates strong performance in high-cutoff ranking metrics, achieving Success@500 of 0.592 and Success@1000 of 0.752, indicating that relevant items are consistently placed within the top portion of the candidate list. At Success@200 and Success@100, VPL outperformed the other two models. However, although VPL outperformed in these two metrics, it is important to note that VPL is not a natural-language conversational baseline in the same sense as CURIO or our agent. Instead, it receives pairwise preference labels over product comparisons, which provide a more explicit supervision signal for latent preference inference. In contrast, our agent and CURIO must infer user preferences from natural-language dialogue, making the task much noisier and more realistic for conversational recommendation. In terms of regret, although VPL achieves a lower average regret (0.0186), our agent attains the lowest average normalized regret (0.0508), suggesting that its recommendations are consistently positioned within high-utility regions relative to the available options. In general, these results indicate that our agent effectively balances preference estimation and ranking performance, producing recommendations that align well with the user’s latent preferences.

5 Conclusion

By combining the concept of uncertainty-reducing intrinsic rewards and a continuous user latent variable vector into one model, our model is able to learn a policy for selecting questions to ask that better uncover the users’ latent variables while simultaneously learning the population average. This not only enables our model to be accurate in predictions by recognizing the population average, but also helps it better personalize a selection for each person with a policy of asking about relevant attributes. By adding in the extrinsic reward as well, we are able to move away from a generic recommendation-to-the-average in a high-density product environment and provide more personalized recommendations for the simulated users.

An interesting next step for this problem would be to move away from supervised training of the belief encoder with MSE loss compared to a “true” latent variable vector. If trained on real world users, defining a “true” latent vector would be difficult if not impossible. We also predefined the set of latent variables for interpretability. One area to explore could be using an autoencoder structure so that the model can find its own latent variables.

References

- [1] W. Pei, Y. Wu, X. Song, et al. Shoppersimulator: Evaluating and exploring rl-driven llm agent for shopping assistants. *arXiv preprint arXiv:2601.18225*, 2026.
- [2] S. Poddar, Y. Wan, H. Ivison, A. Gupta, and N. Jaques. Personalizing reinforcement learning from human feedback with variational preference learning, 2024.
- [3] I. Stelmakh, Y. Luan, B. Dhingra, and M. Chang. Asqa: Factoid questions meet long-form answers. In *Proc. EMNLP*, 2022.
- [4] Y. Wan, J. Wu, M. Abdulhai, L. Shani, and N. Jaques. Enhancing personalized multi-turn dialogue with curiosity reward. *arXiv preprint arXiv:2504.03206*, 2025.
- [5] S. Yao, H. Chen, J. Yang, and K. Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 20744–20757, 2022.
- [6] S. Yao, H. Chen, J. Yang, and K. Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In *ArXiv*, preprint.
- [7] Y. Zhang, S. Zhang, Y. Zhang, et al. Pearl: A personalized large language model framework for e-commerce recommendations. In *Proc. EMNLP Industry Track*, pages 1536–1546, 2024.

Appendix

Appendix A: CURIO Test Results

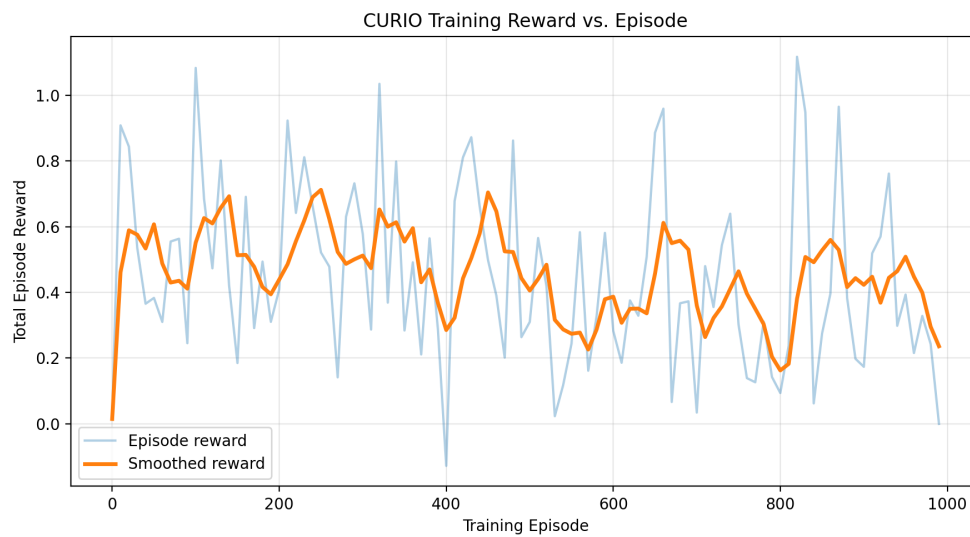


Figure 3: CURIO Training Reward by Episodes

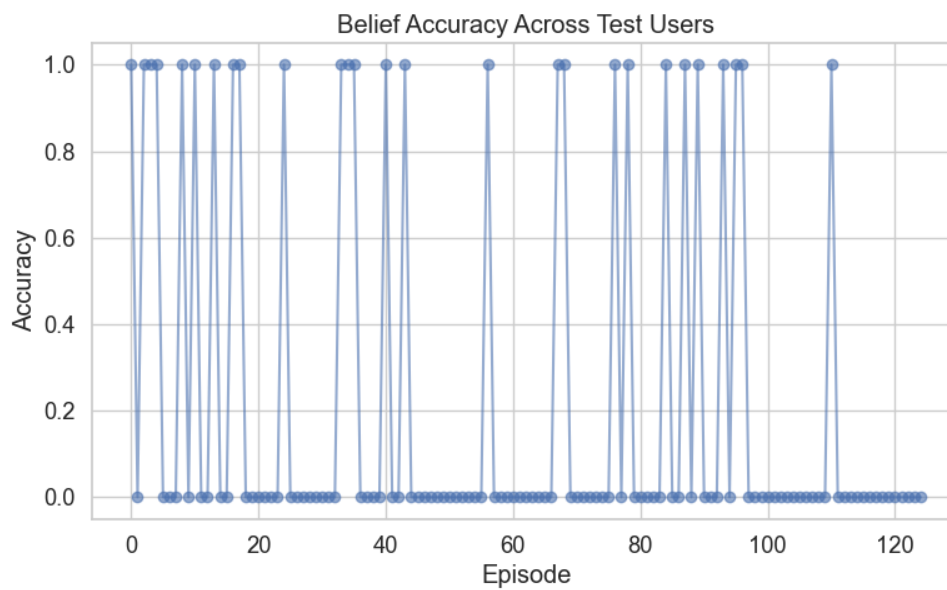


Figure 4: CURIO Belief Accuracy

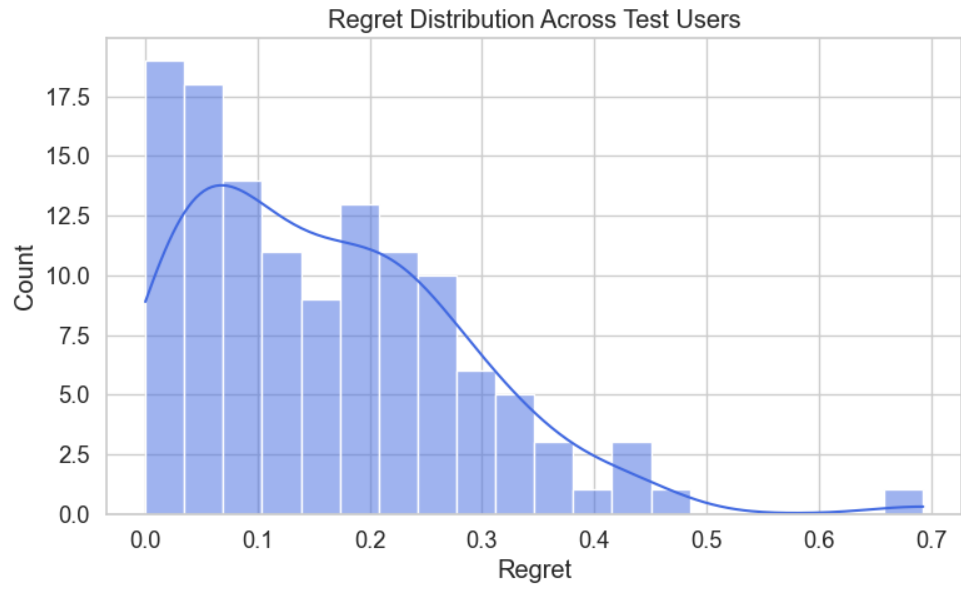


Figure 5: CURIO Regret Distribution

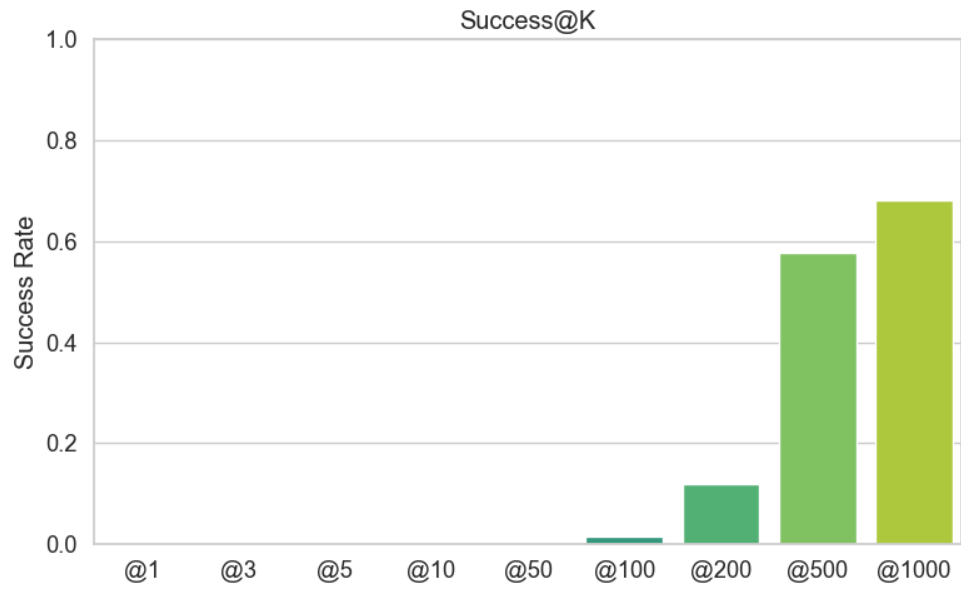


Figure 6: CURIO Success at K

Appendix B: VPL Test Results



Figure 7: VPL Training and Validation Regret by Epoch

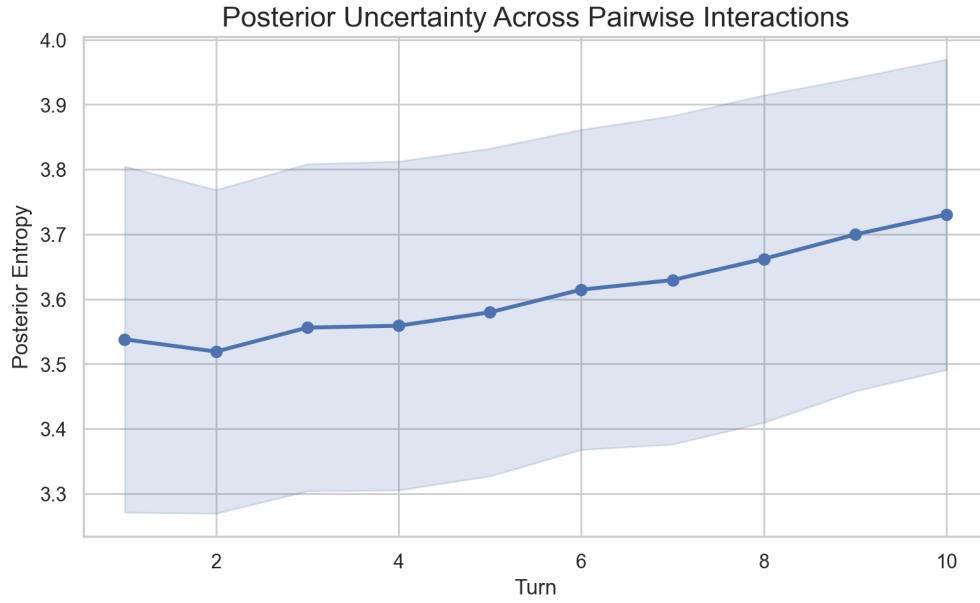


Figure 8: VPL Posterior Uncertainty Reduction Across Pairwise Interactions. The blue curve shows the average posterior entropy after each interaction turn, and the shaded region indicates one standard deviation across test users.

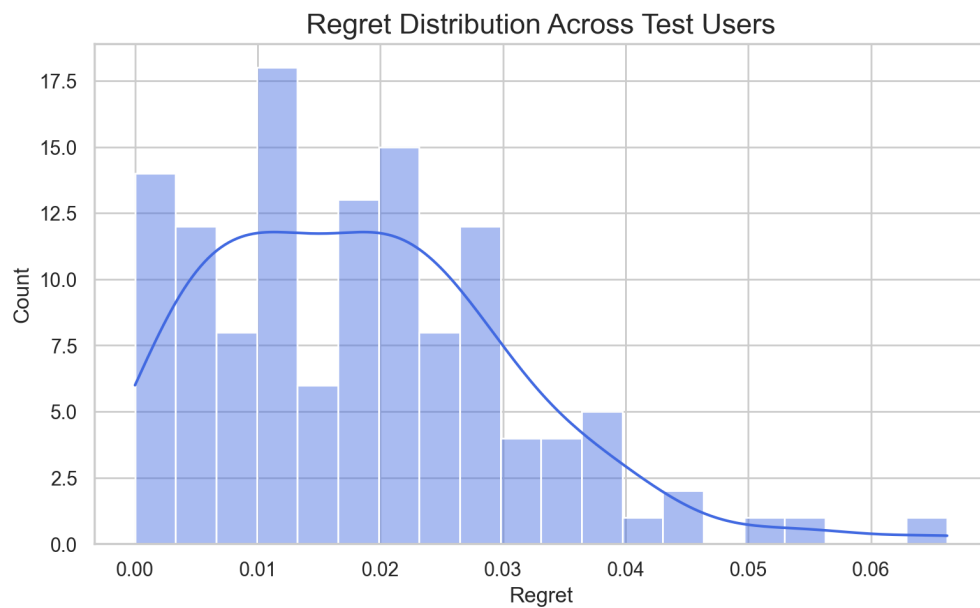


Figure 9: VPL Regret Distribution on the Test Set. The histogram shows the distribution of regret values across test users, and the smooth curve indicates the overall density trend.

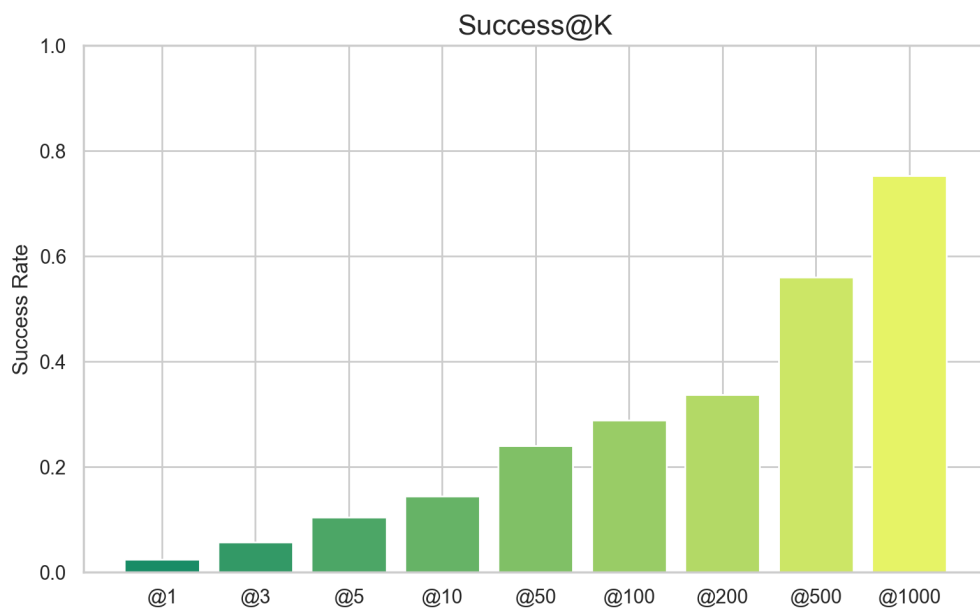


Figure 10: VPL Success at K